

GitHub Skills 第 4 节课柴建铭 2023250361021

柴建铭

2026 年 5 月

GitHub Skills 第 4 节课柴建铭 2023250361021

课程名称：Review pull requests

报告主题：Pull Request 审查与协作反馈

学生姓名：柴建铭

学号：2023250361021

一、核心知识点梳理

本节课学习 Pull Request 审查流程。Pull Request 不只是合并代码的按钮，更是团队成员围绕一次修改进行讨论、检查和改进的协作空间。核心知识包括查看 `changed files`、理解 `diff`、添加评论、提出修改建议、批准或请求更改，以及最终合并。

代码审查的重点不是单纯找错误，而是确认修改是否符合需求、是否影响已有功能、命名是否清晰、代码是否易读、文档是否同步更新。审查者可以在具体代码行下留言，也可以使用 `suggestion` 提出可直接采纳的修改建议。这样既能提高代码质量，也能让开发者理解修改原因，形成团队共享的规范。

二、学习中的难点

学习中的难点是如何进行有效审查。初学时容易只检查代码有没有语法错误，而忽视需求是否满足、边界情况是否考虑、文档是否同步更新等问题。另一个难点是评论表达方式。如果评论过于笼统，例如只写“这里不对”，被审查者很难理解如何修改。较好的方式是指出问题位置、说明原因，并给出可执行的建议。

此外，审查时还需要区分必须修改的问题和可以讨论的建议。不是所有个人偏好都应该强制要求修改，否则会降低协作效率。

三、实践操作与收获

实践过程中，我进入课程提供的 Pull Request，查看 Files changed 页面，对修改内容进行逐行检查。我尝试在具体代码行添加评论，指出命名、格式或逻辑表达方面可以优化的地方，并使用建议修改的方式给出更清晰的文本。随后我根据课程要求完成审查流程，理解评论、请求更改和批准合并之间的区别。

本次实践的收获是认识到 Pull Request 审查可以把个人写代码变成团队共同维护质量的过程。对于软件工程项目来说，审查记录能够保留讨论依据；对于课程小组作业来说，审查可以避免一个人直接把有问题的代码合并到主分支。以后在团队开发中，我需要在提交 PR 时写清楚修改目的、测试情况和影响范围；在审查别人代码时，也要尽量提出具体、客观、可操作的反馈。

四、学习遇到的困惑与疑问（选做）

目前的疑问是：在真实项目中，Pull Request 审查应该设置几个人批准后才能合并比较合适？如果审查者之间意见不一致，应该由谁来决定最终是否合并？

五、报告小结

通过本节 GitHub Skills 课程学习，我不仅完成了平台提供的实践任务，也进一步理解了软件工程中“版本控制、协作开发、文档规范、流程管理”的实际意义。后续学习中，我会继续按照规范提交内容，保持报告结构完整、表达准确、实践过程具体，并在项目训练中主动使用 GitHub 记录每一次关键修改。